

Prompt Engineering Tutorial

Полное руководство по работе с Claude

9 глав | Теория + Практика + Упражнения

Составлено в ходе интерактивной сессии с Claude

Глава 1: Роль (Role Prompting)

Назначение роли меняет стиль, глубину и тон ответа Claude. Это самый простой способ направить модель.

Ключевая идея:

Один и тот же вопрос с разными ролями даёт принципиально разные ответы. Роль задаёт контекст, в котором модель генерирует ответ.

Пример промпта:

"Ты - опытный детский врач-педиатр. Объясни родителям, почему важны прививки."

vs.

"Ты - исследователь вакцин с 20-летним опытом. Объясни механизм работы мРНК-вакцин."

Совет:

Чем конкретнее роль, тем точнее ответ. "Сеньор разработчик с 10 лет опыта" лучше, чем просто "программист".

Глава 2: Тон (Tone)

Тон определяет, КАК модель доносит информацию: формально, дружелюбно, саркастично, академически и т.д.

Ключевая идея:

Тон не меняет содержание, но влияет на восприятие. Один и тот же факт может звучать как лекция профессора или как совет друга.

Пример:

"Отвечай в дружелюбном и ободряющем тоне, как будто ты наставник для джуниора."

Глава 3: Правила и ограничения (Rules)

Правила задают границы поведения модели: что делать, чего избегать, какие ограничения соблюдать.

Ключевая идея:

Чёткие правила предотвращают нежелательное поведение модели. Лучше явно указать ограничения, чем надеяться, что модель сама догадается.

Пример:

"Правила:

- Отвечай только на русском языке
- Не используй жаргон
- Если не знаешь ответ - честно скажи об этом
- Ограничь ответ 3 предложениями"

Глава 4: Примеры (Few-Shot Prompting)

Few-shot prompting - это предоставление модели примеров желаемого ввода/вывода перед основной задачей. Один из самых мощных приёмов.

Ключевая идея:

Примеры показывают модели ФОРМАТ, СТИЛЬ и ЛОГИКУ ожидаемого ответа. Это работает лучше, чем длинные текстовые описания.

Пример:

"Классифицируй отзыв как позитивный или негативный.

Отзыв: Отличный товар, доставили быстро!

Класс: позитивный

Отзыв: Ужасное качество, верну обратно.

Класс: негативный

Отзыв: Всё хорошо, но упаковка помята.

Класс:"

Совет:

2-3 примеров обычно достаточно. Больше примеров = больше стабильность формата.

Глава 5: XML-теги для структуры

XML-теги позволяют чётко разделить разные части промпта: данные, инструкции, формат вывода. Claude отлично понимает XML.

Ключевая идея:

Теги устраняют двусмысленность. Модель точно знает, где данные, где инструкции, и в каком формате отвечать.

Пример:

```
<document>
  текст для анализа...
</document>

<instructions>
  Найди ключевые тезисы в документе выше.
</instructions>

<output_format>
  Ответь в формате нумерованного списка.
</output_format>
```

Популярные теги:

- `<document>`, `<context>` - для входных данных
- `<instructions>` - для задачи
- `<example>` - для примеров
- `<quotes>` - для цитат из источника
- `<thinking>`, `<issues>` - для внутренних размышлений

Глава 6: Формулировка задачи (Task)

Чёткая, конкретная формулировка задачи - основа хорошего промпта. Чем точнее задача, тем точнее результат.

Ключевая идея:

Избегайте расплывчатых формулировок. "Проанализируй код" хуже, чем "Найди потенциальные баги и проблемы с производительностью в этом коде".

Плохо:

"Помоги с кодом."

Хорошо:

"Проведи code-review этой функции. Найди баги, проблемы безопасности и предложи улучшения производительности."

Глава 7: Пошаговое мышление (Chain of Thought)

Инструкция "думай шаг за шагом" заставляет модель рассуждать последовательно, что значительно улучшает точность на сложных задачах.

Ключевая идея:

Без CoT модель может "перескакивать" к ответу, пропуская важные шаги. С CoT она проходит полную цепочку рассуждений.

Пример:

"Реши задачу. Думай шаг за шагом, показывая каждый этап рассуждений."

Задача: В магазине было 45 яблок. Продали 1/3, потом привезли ещё 20. Сколько яблок стало?"

Когда использовать:

- Математические задачи
- Логические рассуждения
- Многошаговый анализ
- Задачи с неочевидным ответом

Глава 8: Формат вывода (Output Format)

Явное указание формата вывода гарантирует, что ответ будет структурирован именно так, как вам нужно.

Ключевая идея:

Модель может отвечать в любом формате: JSON, таблица, список, код, markdown. Но нужно явно об этом попросить.

Пример:

"Ответь строго в формате JSON:

```
{  
  "sentiment": "positive/negative/neutral",  
  "confidence": 0.0-1.0,  
  "reasoning": "краткое обоснование"  
}
```

Глава 9: Построение сложных промптов

Финальная глава объединяет все техники. Сложный промпт может комбинировать любые из 9 элементов в зависимости от задачи.

Все 9 элементов:

- 1. Роль / контекст
- 2. Тон
- 3. Правила

- 4. Примеры (Few-Shot)
- 5. Данные в XML-тегах
- 6. Задача
- 7. Пошаговое мышление (CoT)
- 8. Формат вывода
- 9. Префилл (начало ответа)

Финальное упражнение (Code Review бот):

В качестве итогового задания был построен комплексный промпт, объединивший: роль (сеньор-разработчик), правила (сократический метод), XML-теги (<issues> / <response>), и чёткую задачу (code-review).

Результат: Claude нашёл ZeroDivisionError в функции `calculate_average([])` и задал наводящие вопросы вместо прямых указаний.

Шпаргалка (Cheat Sheet)

Роль	"Ты - [роль] с [опыт]. Твоя задача - ..."
Тон	"Отвечай в [прилагательное] тоне"
Правила	"Правила:\n- Делай X\n- Не делай Y"
Few-Shot	"Пример 1: вход -> выход\nПример 2: вход -> выход\nТеперь: ..."
XML-теги	"<data>...</data> <instructions>...</instructions>"
Задача	"Твоя задача: [конкретное действие] для [цель]"
CoT	"Думай шаг за шагом" / "Размышляй в <thinking>"
Формат	"Ответь в формате [JSON/список/таблица/...]"
Префилл	Начни ответ за модель: "{ \"result\":"

Главное правило:

Не нужно использовать все 9 техник в каждом промпте. Берите только то, что нужно для конкретной задачи. Простая задача = простой промпт. Сложная задача = комбинация техник.